SP – LAB nr 2 - 11.03.2026 - Materiały

📋 Agenda zajęć

Część A: Sprawy Organizacyjne

- Przypomnienie Lab 1 oraz indywidualnych postępów w ramach HW.
- Cel na Lab 2.

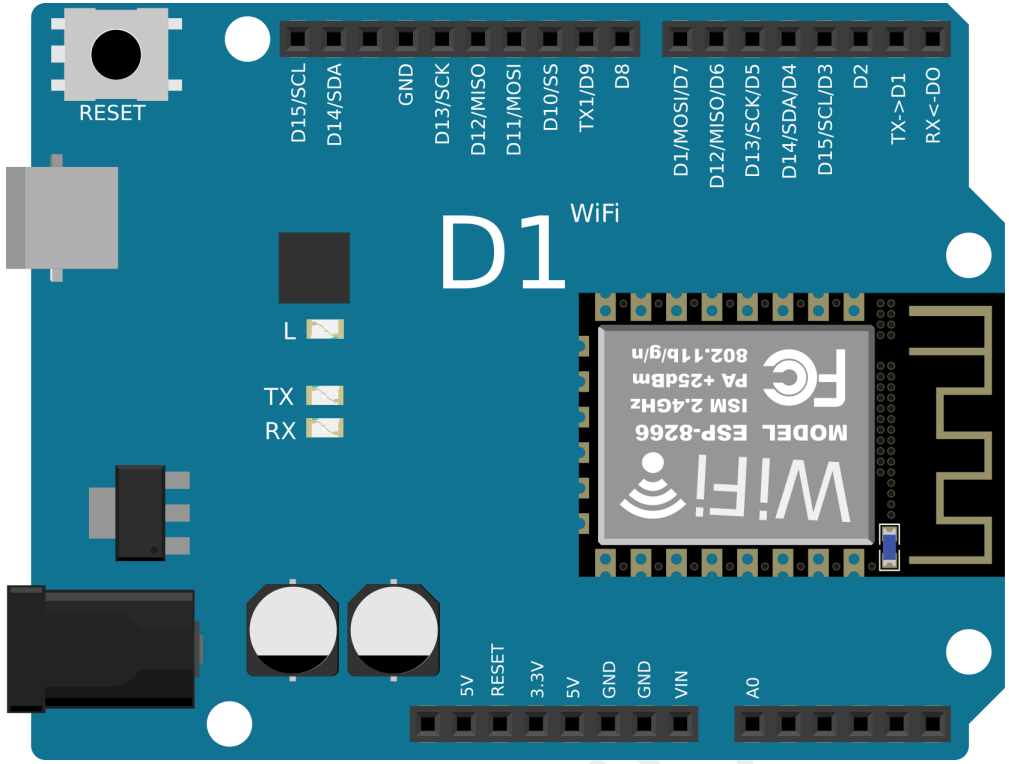
Część B: Warsztat „Brama Powietrzna – I2C & Web Server”

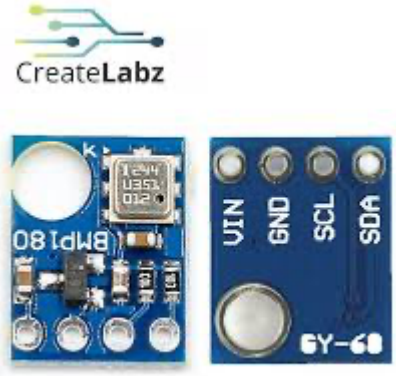
- **Intro:** Omówienie magistrali I2C (SDA/SCL), mapowanie pinów GPIO oraz różnica między trybem WiFi Station a Access Point.
- **Zadanie BASIC:** Fizyczne podłączenie BMP180, diagnostyka magistrali (**I2C Scanner**), odczyt temperatury na Serial oraz wystawienie endpointu `/sensor` serwującego dane w formacie JSON (Temperatura).
- **Zadanie PERFORMANCE:** Rozszerzenie danych o ciśnienie (Serial/HTTP) oraz i implementacja inżynierskiej odporności: obsługa błędów I2C (HTTP 503) i rate-limiting (HTTP 429) z użyciem wzorca **`millis()`**.
- **Check-off & Audit Trail:** Weryfikacja endpointu (Browser/F12), Push kodu na GitHub (tag `lab2`) i czyszczenie poświadczeń Git.

Artefakt: Działający autonomiczny węzeł telemetryczny (Edge Device) z aktywnym serwerem HTTP i diagnostyką sprzętową.

UKW/WI – PPC

🛠️ Projekt: Węzeł Telemetryczny Edge





UKW/WI PPC

🧠 Zanim zaczniesz: Twój Hardware na dziś (Wemos D1 / ESP8266)

Audyt stanowiska: Zanim podłączysz zasilanie, wykonaj „pre-flight check” swojego zestawu. BMP180 to precyzyjny sensor barometryczny komunikujący się po magistrali I2C. Wymaga on rygorystycznego podejścia do poziomów napięć i poprawnego mapowania pinów logicznych na fizyczne etykiety płytki.

1. Schemat połączeń (Pinout Mapping) Podłącz przewody zgodnie z poniższą tabelą. Zwróć uwagę na kolizję oznaczeń VIN – to najczęstszy powód awarii modułów na starcie.

BMP180 (Sensor)	Wemos D1 (Board)	Funkcja / Opis
VIN	**3.3V**	⚠️ **UWAGA:** Nie podłączaj do 5V ani do VIN na płytce!
GND	**GND**	Masa układu (Wspólny potencjał).
SDA	**D4 (GPIO4)**	Serial Data (Linia danych I2C).
SCL	**D3 (GPIO5)**	Serial Clock (Linia zegarowa I2C).

UKW/WI – DPC

2. Inżynierskie pułapki (Hardware Audit)

- **Kolizja etykiet VIN:** Na module BMP180 „VIN” oznacza wejście zasilania sensora (który pracuje w standardzie 3.3V). Na płytce Wemos D1 „VIN” to wejście surowego zasilania z USB (ok. 5V). Połączenie BMP180 VIN do Wemos VIN trwale uszkodzi sensor.
- **Adresowanie I2C:** Fabryczny adres Twojego sensora to `0x77`. Jeśli I2C Scanner go nie widzi, sprawdź jakość styków i poprawność podłączenia linii SDA/SCL.
- **Logika GPIO:** Pamiętaj, że etykieta na laminacie (np. D3) nie jest tożsama z numerem pinu w kodzie (GPIO5). W Twoim `platformio.ini` i kodzie C++ zawsze odwołujemy się do numeracji GPIO lub używamy stałych zdefiniowanych w rdzeniu Arduino.

3. Dokumentacja i Standardy (Verified Rigor)

- Oficjalna dokumentacja magistrali Wire dla ESP8266: arduino-esp8266.readthedocs.io
- Specyfikacja techniczna protokołu I2C (NXP UM10204): nxp.com
- Repozytorium biblioteki sterownika: Adafruit BMP085/BMP180 Library

Złota zasada: „Measure twice, power once”. Błędne podłączenie zasilania do linii sygnałowych I2C natychmiastowo degraduje krzem wewnątrz sensora.

UKW/WI – PPC

🟢 BASIC: (MUST - Wymagane)

Cel: Diagnostyka magistrali I2C, uruchomienie autonomicznego punktu dostępowego (Access Point) oraz wystawienie danych z sensora BMP180 przez protokół HTTP w formacie JSON.

Akcja 1 (Diagnostyka I2C): Podłącz sensor BMP180 (VCC -> 3.3V, GND -> GND, SDA -> D4, SCL -> D3). Wgraj kod **I2C Scanner**. Potwierdź w Serial Monitorze, że urządzenie jest widoczne pod adresem **0x77**.

Akcja 2 (Akwizycja temperatury): Zaimportuj bibliotekę `Adafruit BMP085`. Napisz kod inicjalizujący sensor i wyświetlaj aktualną temperaturę w Serial Monitorze co 2 sekundy.

Akcja 3 (WiFi Soft-AP): Skonfiguruj ESP8266 w trybie punktu dostępowego (`WiFi.mode(WIFI_AP)`). Ustaw SSID jako `Stacja-LAB-NR_INDEXU` oraz hasło `inzynier2026`. Wyświetl adres IP bramy (standardowo `192.168.4.1`) na porcie szeregowym.

Akcja 4 (HTTP Server & JSON): Uruchom `ESP8266WebServer` na porcie 80. Stwórz endpoint `GET /sensor`, który zwraca dokument JSON z temperaturą: `{"temp": 24.5}`. Pamiętaj o ustawieniu typu zawartości na `application/json`.

Akcja 5 (Push): Zrób commit i wypchnij kod na GitHub. Pamiętaj o nadaniu tagu **lab2**.

DoD (Definition of Done):

- ☐ I2C Scanner wykrywa sensor pod adresem 0x77.
- ☐ Płytka rozgłasza sieć WiFi z zadanym SSID.
- ☐ Po połączeniu z WiFi i wejściu w przeglądarce na `/sensor` wyświetla się poprawny JSON z temperaturą.
- ☐ Repozytorium na GitHubie zawiera poprawną strukturę i tag `lab2`.

UKW/WI - PPC

🟡 PERFORMANCE: (dla szybszych)

Cel: Rozszerzenie telemetrii o ciśnienie, implementacja inżynierskiej odporności poprzez obsługę kodów błędów HTTP oraz eliminacja blokowania procesora.

Akcja 1 (Rozszerzona Akwizycja): Dodaj odczyt ciśnienia (`pressurePa`) do logów Serial Monitora oraz do payloadu JSON zwracanego przez serwer.

Akcja 2 (Obsługa błędów - HTTP 503): Zaimplementuj mechanizm sprawdzania stanu sensora. Jeśli komunikacja I2C zostanie zerwana, endpoint `/sensor` musi przestać wysyłać stare dane i zacząć zwracać kod **503 Service Unavailable**.

Akcja 3 (Rate-limiting - HTTP 429): Wprowadź blokadę zbyt częstych zapytań. Jeśli klient odświeża endpoint częściej niż raz na 2 sekundy, serwer ma zwrócić kod **429 Too Many Requests**.

Akcja 4 (Deterministyczna pętla): Zmodyfikuj kod tak, aby obsługa serwera i odczytów odbywała się bez użycia funkcji `delay()`. Wykorzystaj funkcję **`millis()`** do zarządzania interwałami czasowymi.

DoD:

- ☐ JSON zawiera klucze `temp` oraz `pressurePa`.
- ☐ Fizyczne odłączenie linii SDA/SCL powoduje, że serwer zwraca błąd 503.
- ☐ Zbyt szybkie odświeżanie strony (F5) skutkuje błędem 429.
- ☐ Kod udowadnia brak funkcji `delay()` w głównej pętli `loop()`.

UKW/WI – PPC

🔵 DELIGHTERS (dla ambitnych)

Cel: Implementacja wizualnego feedbacku, zaawansowana metrologia oraz utwardzenie bezpieczeństwa węzła (Security Hardening).

Akcja 1 (Feedback LCD): Podłącz wyświetlacz LCD 2x16 (I2C). Wyświetlaj na nim temperaturę, ciśnienie oraz aktualną liczbę obsłużonych żądań HTTP.

Akcja 2 (Estymacja wysokości): Wykorzystaj dane o ciśnieniu do wyliczenia przybliżonej wysokości nad poziomem morza (`readAltitude()`). Wynik dodaj do wyświetlacza LCD oraz do odpowiedzi JSON.

Akcja 3 (Observability): Przy każdym poprawnym żądaniu GET loguj na port szeregowy adres IP klienta oraz czas systemowy (uptime) wykonania zapytania.

Akcja 4 (Security Hardening): 1. Ukryj SSID swojej sieci WiFi (Hidden SSID). 2. Wprowadź prostą autoryzację: serwer ma zwracać dane tylko, gdy w zapytaniu obecny jest nagłówek `X-API-KEY` z poprawnym kluczem. W innym przypadku zwróć **401 Unauthorized**.

Akcja 5 (Optymalizacja JSON): Zastąp ręczne budowanie ciągów znaków (stringów) profesjonalną biblioteką **ArduinoJson** (użyj `StaticJsonDocument`).

DoD:

- ☐ Wartości temperatury, ciśnienia i wysokości są widoczne na LCD.
- ☐ Serwer loguje IP klienta przy każdym wejściu.
- ☐ Dostęp do danych jest chroniony nagłówkiem API-Key.
- ☐ Kod wykorzystuje bibliotekę ArduinoJson do generowania odpowiedzi.

UKW/WI - PPC

⚡ Cheat Sheet

I. GPIO: Programowalne "nogi" procesora

I. Architektura Sieciowa ESP8266

Station (STA) vs Access Point (AP) W systemach IoT węzeł brzegowy (Edge Node) musi umieć dostosować się do dostępnej infrastruktury. Układ radiowy ESP8266 pozwala na sprzętową rekonfigurację topologii w dwóch głównych trybach:

- Station (STA): Węzeł pracuje jako klient. Skanuje pasmo w poszukiwaniu zewnętrznego routera (infrastruktury), autoryzuje się i pobiera adres IP (np. przez DHCP).
- Access Point (AP / Soft-AP): Węzeł przejmuje rolę infrastruktury (routera). Samodzielnie rozgłasza sieć, przydziela adresy IP podłączonym klientom i scentralizowanie kontroluje ruch sieciowy.

UKW/WI – PPC

W warunkach laboratoryjnych (brak zewnętrznego routera) wymuszamy architekturę AP. Płytka staje się całkowicie autonomicznym serwerem.

Inżynierska Implementacja Soft-AP: Aby zarządzać stosem sieciowym, musisz zaimportować dedykowaną bibliotekę rdzenia i wywołać metody konfigurujące radio. Złota zasada Soft-AP: W tym trybie węzeł przyjmuje sztywny, domyślny adres IP bramy: 192.168.4.1. To jest punkt kontaktowy dla Twojej przeglądarki i skryptów testowych.

```
#include <ESP8266WiFi.h> // Wymagany nagłówek do obsługi stosu TCP/IP i radia

// ... wewnątrz funkcji setup() ...

// 1. Definicja trybu pracy radia
WiFi.mode(WIFI_AP);

// 2. Inicjalizacja sieci bazowej
WiFi.softAP("Stacja-LAB-NR_INDEXU", "inzynier2026");
```

Słownik Metod API:

- `WiFi.mode(WIFI_AP)`: Instrukcja dla sprzętowego kontrolera radiowego. Jawnie odcina logikę klienta (STA), zwalniając zasoby procesora (blokuje ciągłe skanowanie w tle w poszukiwaniu innych sieci). Zwiększa to determinizm i wydajność działania głównej pętli programu.
- `WiFi.softAP(ssid, password)`: Uruchamia rozgłaszanie sieci. Przyjmuje dwa ciągi znaków. Użycie parametru `password`(minimum 8 znaków) automatycznie wymusza sprzętowe szyfrowanie WPA2-PSK w warstwie łącza danych. Podanie tylko `ssid` utworzyłoby sieć otwartą (antywzorzec bezpieczeństwa).

UKW/WI – PPC

II. GPIO

Programowalne "nogi" procesora GPIO (General Purpose Input/Output) to uniwersalne piny mikrokontrolera. Pracują one w dwóch warstwach, co często jest pułapką dla początkujących:

1. Warstwa fizyczna (Etykiety): To, co widzisz wydrukowane na laminacie płytki (np. D3, D4).
2. Warstwa logiczna (Krzem): Rzeczywisty numer pinu w rejestrach procesora (np. GPIO4, GPIO5).

Inżynierska pułapka: Kompilator C++ (framework Arduino) domyślnie operuje na numerach GPIO, a nie na etykietach D1/D2 (chyba że użyjesz wbudowanych stałych). Na płytkach rodziny Wemos D1 R1 mapowanie wygląda następująco:

- D4 to fizycznie GPIO4.
- D3 to fizycznie GPIO5.

III. Magistrala I2C i Fizyka Połączenia I2C (Inter-Integrated Circuit)

I2C to potężna szyna komunikacyjna (Master-Slave) wykorzystująca zaledwie dwa przewody sygnałowe. ESP8266 (Master) odpytuje sensor BMP180 (Slave) po jego unikalnym adresie sprzętowym (0x77).

- SDA (Serial Data): Linia przesyłowa danych (podłącz do D4).
- SCL (Serial Clock): Linia zegarowa, synchronizująca transmisję (podłącz do D3).

KRYTYCZNE: Zasilanie. Piny sygnałowe I2C tolerują tylko napięcie 3.3V. Zasil sensor BMP180 wyłącznie z pinu 3.3V na płytce. Podłączenie do pinu VIN (który przenosi 5V z USB) natychmiastowo zniszczy sensor.

UKW/WI – PPC

IV. Diagnostyka poprawności podłączenia oraz sprawności BMP 180

Poniżej masz I2C Scanner (Diagnostyka Sprzętu). Zasada: nigdy nie pisz logiki aplikacji, dopóki nie udowodnisz, że warstwa fizyczna działa. Poniższy kod to Twój radar. Wgraj go przed rozpoczęciem zadań. Jeśli w konsoli nie zobaczysz "0x77", masz problem z kablami, brakiem styku na płytce stykowej lub zasilaniem.

```
#include <Arduino.h>
#include <Wire.h>

void setup() {
  Serial.begin(115200);
  delay(2000); // Czas na ustabilizowanie monitora

  // Jawne przypisanie pinów I2C: SDA = GPIO4 (D4), SCL = GPIO5 (D3)
  Wire.begin(4, 5);

  Serial.println("\n--- Rozpoczynam skanowanie I2C ---");
}
```

UKW/WI – PPO

```
void loop() {
  byte error, address;
  int devicesFound = 0;

  for(address = 1; address < 127; address++) {
    Wire.beginTransmission(address);
    error = Wire.endTransmission();

    if (error == 0) {
      Serial.print("Znaleziono urządzenie I2C pod adresem: 0x");
      Serial.println(address, HEX);
      devicesFound++;
    }
  }

  if (devicesFound == 0) {
    Serial.println("KRYTYCZNE: Brak urządzeń. Sprawdź kable SDA/SCL i zasilanie 3.3V.");
  }

  delay(5000);
}
```

Wskazówka: Zauważ jawne wymuszenie `Wire.begin(4, 5)`. Bez tego framework mógłby założyć domyślne piny dla innej wersji płytki. Jako inżynierowie nie pozostawiamy takich rzeczy przypadkowi.

UKW/WI – PPC

V. Konfiguracja Środowiska (platformio.ini)

Aby pisać kod dla BMP180 oraz (później) serwować profesjonalny JSON, musimy dodać zewnętrzne zależności. Twój `platformio.ini` powinien zawierać blok `lib_deps`:

```
[env:d1]
platform = espressif8266
board = d1
framework = arduino
monitor_speed = 115200

lib_deps =
    adafruit/Adafruit BMP085 Library
    bblanchon/ArduinoJson ; Zostaw na zadanie DELIGHTERS
```

VI. API Sensora BMP180 - Odczyt Danych

Po zainstalowaniu biblioteki, zyskujesz dostęp do gotowych funkcji przeliczających surowe bity na jednostki układu SI.

1. Inicjalizacja: W global scope tworzysz obiekt: `Adafruit_BMP085 bmp;`
2. Start: W funkcji `setup()` sprawdzasz, czy sensor "żyje": `bmp.begin()` (zwraca `true` lub `false`). To Twój bezpiecznik dla fail-safe.
3. Odczyt: W pętli używasz dedykowanych metod. Pamiętaj o typach danych:
    - `float temp = bmp.readTemperature();` (Zwraca °C)
    - `int32_t press = bmp.readPressure();` (Zwraca Paskale)
    - `float alt = bmp.readAltitude();` (Estymuje wysokość w metrach, na bazie ciśnienia)

UKW/WI – PPC

VII. Protokół HTTP i Kody Statusów (Sygnalizacja Błędów)

Budujesz serwer (`ESP8266WebServer server(80);`), który musi gadać z klientem (przeglądarką) za pomocą standardu HTTP. Aby nasłuchiwać zapytań, w funkcji `setup()` musisz powiązać ścieżkę (endpoint) z logiką:

```
server.on("/sensor", []() {
    // Logika weryfikacji i wysyłania odpowiedzi
});
server.begin();
```

Złota zasada API: Sama odpowiedź tekstowa nie wystarczy. Musisz poinformować klienta o statusie operacji. Do tego służą kody HTTP, które przekazujemy w funkcji `server.send(kod, typ_danych, zawartosc)`:

- 200 (OK): Wszystko działa. Dane lecą do klienta.
- 503 (Service Unavailable): Użyj go, jeśli zaimplementujesz fail-safe i odkryjesz, że sensor jest odłączony. Informujesz klienta: "Węzeł żyje, ale sensor jest martwy".
- 429 (Too Many Requests): Ochrona infrastruktury. Ktoś spamuje serwer żądaniami szybciej niż raz na 2 sekundy? Zwróć ten kod jako odrzucenie żądania.

💡 **Ważne**: Ustawienie nagłówka `Content-Type`. Jeśli wysyłasz JSON, drugi parametr w `server.send` MUSI wynosić `"application/json"`. Jeśli ustawisz `"text/plain"`, profesjonalne systemy odrzucą Twój payload.

UKW/WI – PPC

Ręczne budowanie payloadu JSON (Konkatenacja Stringów - Wzorzec BASIC): Na wczesnym etapie budowy systemu (zanim użyjesz profesjonalnych serializatorów) dokument JSON składasz ręcznie. Wymaga to użycia znaków ucieczki (`\"`), aby kompilator poprawnie zinterpretował cudzysłowy wewnątrz ciągu znaków.

```
float temp = bmp.readTemperature();

String json = "{";
json += "\"temp\": ";
json += String(temp, 2); // 2 miejsca po przecinku
json += "}";

server.send(200, "application/json", json);
```

⁉ **Inżynierska pułapka:** Ręczne operacje na obiekcie `String` (używanie operatora `+=`) przy każdym zapytaniu HTTP alokują nową pamięć. W systemach pracujących 24/7 prowadzi to do fragmentacji sterty (Heap Fragmentation) i zawieszenia mikrokontrolera. Dlatego standardem branżowym (omówionym w sekcji XII) jest użycie biblioteki `ArduinoJson`.

UKW/WI – PPC

VIII. Jak testować działanie serwera? (Browser / curl)

Stworzyłeś endpoint `/sensor`? Nie wierz kodowi na słowo. Musisz to przetestować.

Metoda 1:

Web Browser i F12 (DevTools) Połącz się z siecią WiFi swojej płytki. Wpisz w przeglądarkę `http://192.168.4.1/sensor`. Naciśnij F12 na klawiaturze i przejdź do zakładki Network (Sieć). Odśwież stronę. Kliknij na swoje zapytanie na liście. Zobaczysz dokładne Headers (Nagłówki), w tym kod statusu (np. 200, 503) oraz surowy Response (Twojego JSON-a).

Metoda 2:

Narzędzie curl (Standard inżynierski - CLI) Przeglądarka ukrywa wiele detali. Prawdziwi inżynierowie testują API z poziomu terminala. Jeśli pracujesz w MSys2, Git Bash lub na Linuxie, użyj: `curl -v http://192.168.4.1/sensor`.
Flaga `-v` (verbose) wymusi na konsoli wyrzucenie całego logu negocjacji TCP/IP, requestu i surowych nagłówków zwrotnych. Tam brutalnie zobaczysz każdy błąd 401, 429 czy 503, nawet bez renderowania HTML.

IX. Observability: Diagnostyka Żądań HTTP

Zanim wyślesz JSON, musisz wiedzieć, kto o niego prosi. Zrzucanie danych w próżnię bez ich ewidencjonowania to błąd architektoniczny. Twój serwer musi logować przychodzący ruch sieciowy. Wewnątrz funkcji obsługującej zapytanie (np. handlerze dla `/sensor`) wygeneruj ślad diagnostyczny na port szeregowy:

```
Serial.print("[HTTP GET /sensor] IP Klienta: ");
Serial.print(server.client().remoteIP());
Serial.print(" | Uptime serwera: ");
Serial.print(millis());
Serial.println(" ms");
```

UKW/WI - PPC

Dzięki temu zyskujesz pełną transparentność operacji – widzisz dokładnie adres IP urządzenia wykonującego zapytanie oraz moment w czasie (uptime), w którym to nastąpiło.

X. Bezpieczeństwo

W inżynierii IoT system, który "tylko działa", jest systemem podatnym. Wdrażamy warstwy ochrony, podnosząc stopień skomplikowania ataku.

1. Ukrycie sieci (Hidden SSID): Zmodyfikuj inicjalizację punktu dostępowego. Dodanie parametrów ukryje nazwę sieci przed skanowaniem (Security through obscurity):

```
// Kanał 1, hidden = true (1)
WiFi.softAP("Stacja-LAB-NR_INDEXU", "inzynier2026", 1, 1);
```

2. Autoryzacja nagłówkiem (API-Key): Wymuś na kliencie udowodnienie tożsamości. Aby odczytać z serwera pożądany nagłówek, musisz najpierw poinstruować instancję `ESP8266WebServer`, by go "zbierała". Dodaj to w `setup()`:

```
const char* headerKeys[] = {"X-API-KEY"};
server.collectHeaders(headerKeys, 1);
```

3. Wewnątrz funkcji handlera zaimplementuj twardą weryfikację. Jeśli klucz się nie zgadza, natychmiast utnij komunikację:

```
if (server.header("X-API-KEY") != "tajne-haslo-123") {
    server.send(401, "text/plain", "401 Unauthorized");
    return; // Odcięcie wykonania
}
// ... tutaj wysyłasz JSON 200 OK
```

UKW/WI – PPC

XI. Determinizm i Współbieżność - Wzorzec millis()

W systemach sieciowych użycie `delay()` to sabotaż architektoniczny. Usypia to główny wątek mikrokontrolera, powodując, że serwer WWW (stos TCP/IP) przestaje odpowiadać na żądania klientów. Złota zasada: Zamiast zatrzymywać czas, mierz go. Do ochrony przed spamem (Rate-limiting HTTP 429) oraz do okresowych odczytów sensora używaj funkcji `millis()`, która zwraca czas od startu procesora.

```
unsigned long previousMillis = 0;
const long interval = 2000;

void loop() {
  server.handleClient(); // Nasłuchiwanie HTTP musi działać w tle bez blokad!

  unsigned long currentMillis = millis();
  if (currentMillis - previousMillis >= interval) {
    previousMillis = currentMillis;
    // ... Miejsce na Twój kod wykonujący się dokładnie co 2 sekundy
  }
}
```

UKW/WI – PPC

XII. Interfejs HMI: Wyświetlacz LCD 2x16 I2C

Systemy wbudowane często wymagają lokalnego monitorowania stanu. Wyświetlacz LCD z konwerterem I2C podłączasz do tej samej szyny co BMP180 (SDA -> D4, SCL -> D3). Urządzenia nie będą na siebie "wpadać", ponieważ mają inne adresy sprzętowe.

- Adresacja: Standardowy adres dla modułów z konwerterem PCF8574 to `0x27` (jeśli nie działa, użyj I2C Scannera z sekcji IV!).
- Inicjalizacja (wymaga biblioteki `LiquidCrystal_I2C`):

LiquidCrystal_I2C lcd(0x27, 16, 2); // W global scope

```
// W setup():
lcd.init();
lcd.backlight();
lcd.setCursor(0, 0);
lcd.print("Brama Aktywna");
```

UKW/WI – PPC

XIII. Profesjonalna Serializacja - ArduinoJson

W podstawowej wersji serwera możesz złożyć JSON-a ręcznie przez łączenie stringów (konkatenacja). Jednak przy dużych danych prowadzi to do fragmentacji pamięci sterty (Heap Fragmentation) i wycieków pamięci (Memory Leak). Standardem branżowym na mikrokontrolerach jest biblioteka `ArduinoJson`.

```
#include <ArduinoJson.h>

// Bufor alokowany w pamięci statycznej - eliminuje wycieki i fragmentację
StaticJsonDocument<256> doc;
doc["temp"] = 24.5;
doc["pressurePa"] = 101325;
doc["altitude"] = 65.2;

String response;
serializeJson(doc, response);
server.send(200, "application/json", response);
```

UKW/WI – PPO

📚 Źródła i linki

Jeśli utknąłeś, nie "zgaduj". Inżynier zagląda do specyfikacji:

Poziom Zadania	Rozwiązywany Problem	Link ratunkowy (Source of Truth)
🟢 **BASIC**	Konfiguracja Soft-AP i Serwera WWW	Dokumentacja ESP8266 Arduino Core -> Wyszukaj: *Soft Access Point* oraz *ESP8266WebServer*. Fundament działania stosu sieciowego.
🟢 **BASIC** / 🟡 **PERFORMANCE**	Odczyt danych z BMP180	Repozytorium GitHub -> Wyszukaj: *Adafruit-BMP085-Library*. Czytaj kody źródłowe nagłówków (`.h`), by poznać precyzyjne zwracane typy danych (np. `float`, `int32_t`).
🟡 **PERFORMANCE**	Znaczenie kodów HTTP (503, 429)	MDN Web Docs (Mozilla) -> Wyszukaj: *HTTP response status codes*. Globalny standard sygnalizacji błędów w nowoczesnych API.
🔵 **DELIGHTER**	Profesjonalna serializacja JSON	Dokumentacja biblioteki -> *ArduinoJson.org*. Skorzystaj z wbudowanego narzędzia *Assistant*, aby wyliczyć dokładny rozmiar bufora dla `StaticJsonDocument`.
🔵 **DELIGHTER**	Obsługa ekranu po szynie I2C	Rejestr bibliotek PlatformIO -> Szukaj: *LiquidCrystal_I2C*. Dokumentacja powie Ci, jak zainicjalizować ekran. Pamiętaj sprawdzić na pinoucie płytki, które piny to sprzętowe SDA i SCL.

UKW / WI

🎯 Na kolejny lab (LAB03 - Cyfrowy Bliźniak i Symulacja WEB)

Cel: Oderwanie logiki biznesowej od fizycznego krzemu. Wprowadzimy wirtualne środowisko symulacyjne (Tinkercad/Wokwi), aby umożliwić dewelopment i testowanie systemów IoT bez noszenia hardware'u w plecaku.

1. **Co musisz dokończyć w domu?** Zadania BASIC są obowiązkowe – węzeł musi serwować surowy JSON. Zadania PERFORMANCE (inżynierska odporność na błędy 503/429) są rekomendowane - traktuj jako swój zawodowy standard. Ścieżka DELIGHTER (utwardzanie bezpieczeństwa i profesjonalna serializacja) jest opcjonalna, ale silnie rekomendowana dla inżynierów aspirujących do projektowania systemów klasy produkcyjnej oraz chcących pogłębić aspekty bezpieczeństwa. Jeśli nie zamknąłeś/zamknąłeś sekcji BASIC podczas warsztatu, traktuj to jako dług techniczny do spłaty przed następnym spotkaniem.
2. **Zapowiedź: Co będziemy budować?** Przeniesiemy nasz fizyczny węzeł do przestrzeni wirtualnej. Stworzymy tzw. "Cyfrowego Bliźniaka" (Digital Twin). Zastosujemy ten sam kod C++ i utrzymamy ten sam kontrakt JSON, ale całość uruchomimy z poziomu symulatora w przeglądarce. Udowodnimy sobie, że rygorystyczny kod Embedded można pisać i testować w pociągu, nie mając przy sobie lutownicy, kabli ani czujnika na biurku.
3. **Zastanów się nad bezpieczeństwem:** Spójrz na swój działający serwer (poziom PERFORMANCE)i zastanów się, co mogłoby pójść nie tak w prawdziwym środowisku. W jaki sposób złośliwy użytkownik w tej samej sieci Wi-Fi mógłby zmanipulować Twoje dane lub zablokować działanie urządzenia? Pomyśl, co musisz zmienić w kodzie, aby węzeł przetrwał takie uderzenie.

Widzimy się za dwa tygodnie. Znajdźcie sposób, by wasz kod był niezawodny! 💪

UKW / WI - SPC